INSTITUTE OF COMPUTER TECHNOLOGY

B-TECH COMPUTER SCIENCE ENGINEERING 2025-26

SUBJECT: CRYPTOGRAPHY

NAME: Rahul Prajapati

ENRLL. NO: 23162171020

BRANCH: CYBER SECURITY

BATCH: 52

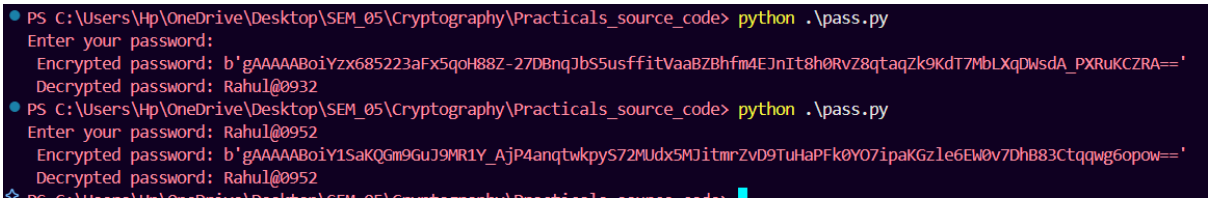
PRACTICAL_1

Aim: To understand the fundamentals of encryption and decryption by implementing a basic or standard encryption algorithm that takes a user-inputted password, converts it into ciphertext, and then decrypts it back to its original form, displaying both ciphertext and original text in the console.

Source_Code:

```
from cryptography.fernet import Fernet
import getpass
key = Fernet.generate_key()
cipher = Fernet(key)
password = getpass.getpass("Enter your password: ").encode()
encrypted = cipher.encrypt(password)
print(" Encrypted password:", encrypted)
decrypted = cipher.decrypt(encrypted)
print(" Decrypted password:", decrypted.decode())
```

OUTPUT:-



INSTITUTE OF COMPUTER TECHNOLOGY

B-TECH COMPUTER SCIENCE ENGINEERING 2025-26

SUBJECT: CRYPTOGRAPHY

NAME: Rahul Prajapati

ENRLL. NO: 23162171020

BRANCH: CYBER SECURITY

BATCH: 52

PRACTICAL_2

Aim:-To compare various encryption methods used by classmates by exchanging ciphertexts and attempting decryption using both their original decryption logic and third-party tools, in order to evaluate the effectiveness and reversibility of different encryption techniques.

Method_1:

Code:

```python
import hashlib
passwd = input("Enter password: ")

encoded_passwd = passwd.encode('utf-8')
hashed_passwd = hashlib.sha512(encoded_passwd).hexdigest()

print(f"Original password (encoded): {encoded_passwd}")
print(f"Hashed password (md5): {hashed_passwd}")

usr_passwd = input("Re-enter password: ")

if hashlib.sha512(usr_passwd.encode('utf-8')).hexdigest() == hashed_passwd:
    print(f"decrypted password:{usr_passwd}")
else:
    print("Password does not match.")
```

Output:

```
PS C:\Users\Hp\OneDrive\Desktop\SEM_05\Cryptography\Practicals_source_code> & C:/Python313/python.exe c:/Users/Hp/OneDrive/Desktop/SEM_05/Cryptography/Prac
de/practical2_1.py
Enter password: rahul@12
Original password (encoded): b'rahul@12'
Hashed password (md5): 20b383ac020ea7b1722792b4b4f0e59ea55f6f452335d87486621cdef9d5608aeeb918fb62e25fff2e13a2eabd798f339ca9bc9f72e494127e12eadf47aff423
Re-enter password: rahul@12
hash matched successfully
decrypted password:rahul@12
```

Method_2:

Code:-

```python
from cryptography.fernet import Fernet
import getpass

key = Fernet.generate_key()
cipher = Fernet(key)

password = input("Enter your password: ").encode()

encrypted = cipher.encrypt(password)
print(" Encrypted password:", encrypted)

decrypted = cipher.decrypt(encrypted)
print(" Decrypted password:", decrypted.decode())
```

Output:-

```
de/practical2_2.py
Enter your password: hello
 Encrypted password: b'gAAAAABomFeNNvDNxLDV-kHeOGDGXKYsl6dpLX-yMtuCuZXaJrjN8aNNuZkowGEOjMgYAcJQBlYJQSOtzRdHQh1NsfU6-FpFrg=='
 Decrypted password: hello
```

Method_3:

Code:

```python
def caesar_cipher(text, shift, mode='encrypt'):
    if mode == 'decrypt':
        shift = -shift
    processed_text = ""
    for char in text:
        if char.isalpha():
            shift_amount = shift % 26
            if char.islower():
                start = ord('a')
                processed_char = chr(start + (ord(char) - start + shift_amount) % 26)
            else:
                start = ord('A')
                processed_char = chr(start + (ord(char) - start + shift_amount) % 26)
            processed_text += processed_char
        else:
            processed_text += char
    return processed_text
text = input("Enter the text: ")
shift = int(input("Enter the shift value: "))
mode = input("Enter 'encrypt' to encrypt or 'decrypt' to decrypt: ")
result = caesar_cipher(text, shift, mode)
print(f"Result: {result}")
```

Output:

```
de/practical2_3.py
Enter the text: rahulhello
Enter the shift value: 3
Enter 'encrypt' to encrypt or 'decrypt' to decrypt: encrypt
Result: udkxokhoor
PS C:\Users\Hp\OneDrive\Desktop\SEM_05\Cryptography\Practicals_source_code> & C:/Python313/python.exe c:/U
de/practical2_3.py
Enter the text: udkxokhoor
Enter the shift value: 3
Enter 'encrypt' to encrypt or 'decrypt' to decrypt: decrypt
Result: rahulhello
```

1. Analysis – Which was stronger and when it failed

Method 1 – SHA512 "encryption"

- Here we take the password, turn it into a fixed scrambled code.
- When we "decrypt" in your code, we're actually just checking if the scrambled version matches the original scrambled one.
- In your example, it works fine if the password is correct, but fails instantly if it's even 1 letter wrong.
- Strength-wise, it's much harder to break than Caesar Cipher, but weaker than Fernet if you actually need to get the original data back.

Method 2 – Fernet encryption

- Proper encryption. You can lock (encrypt) and unlock (decrypt) perfectly if you have the key.
- In your code, it worked every time because the same key was used right away.
- Security is very high, but only if you keep the key safe.

Method 3 – Caesar Cipher

- Very old-school and weak.
- Anyone can break it in seconds using trial and error or an online decoder.
- It still works for both encrypt and decrypt in the code, but not safe at all in real life.

2. Summary – Why this matters in real life

- **Most secure overall:** Method 2 (Fernet) – solid protection and can get your data back.
- **Good for checking passwords but not for actual data retrieval:** Method 1.
- **Fun but useless for real protection:** Method 3.

- In real life, if you choose a weak method like Caesar Cipher, it's like locking your phone with "1234" anyone can open it.
- If you use Fernet or a strong hash, it's like having a fingerprint lock plus a PIN.

3. Limitations

Method 1 – SHA512

- **Easy to break?** Not really, but weak passwords still can be guessed with tools.
- **Key problem?** No key, so you can't "actually" decrypt — only compare.
- **Data type?** Works for text but not great for other data like images unless converted first.
- **Real life?** Good for storing and checking passwords, not for retrieving original stuff.

Method 2 – Fernet

- **Easy to break?** Nope, unless someone gets your key.
- **Key problem?** Lose the key = lose the data forever.
- **Data type?** Works with any type of data.
- **Real life?** Perfect for protecting files, emails, or any sensitive info.

Method 3 – Caesar Cipher

- **Easy to break?** Yep, super easy.
- **Key problem?** The “key” (shift number) is so small it can be guessed in seconds.
- **Data type?** Only works properly with letters.
- **Real life?** Only for games or secret notes between kids.

INSTITUTE OF COMPUTER TECHNOLOGY

B-TECH COMPUTER SCIENCE ENGINEERING 2025-26

SUBJECT:CRYPTOGRAPHY

NAME: Rahul Prajapati

ENRLL. NO: 23162171020

BRANCH: CYBER SECURITY

BATCH: 52

PRACTICAL_03

Aim: To implement both the classic Caesar Cipher (fixed shift value of 3) and a custom Shift Cipher (user-defined shift value) in Python, display their encrypted and decrypted outputs, and analyze the security of the shift cipher using brute-force attacks.

CODE:

```python
def encrypt_caesar(text):
    result = ""
    for char in text:
        if char.isalpha():
            ascii_offset = ord('A') if char.isupper() else ord('a')
            shifted = (ord(char) - ascii_offset + 3) % 26 + ascii_offset
            result += chr(shifted)
        else:
            result += char
    return result

def decrypt_caesar(text):
    result = ""
    for char in text:
        if char.isalpha():
            ascii_offset = ord('A') if char.isupper() else ord('a')
            shifted = (ord(char) - ascii_offset - 3) % 26 + ascii_offset
            result += chr(shifted)
        else:
            result += char
    return result

def encrypt_shift(text, shift):
    result = ""
    for char in text:
        if char.isalpha():
            ascii_offset = ord('A') if char.isupper() else ord('a')
            shifted = (ord(char) - ascii_offset + shift) % 26 + ascii_offset
            result += chr(shifted)
        else:
            result += char
    return result

def decrypt_shift(text, shift):
    return encrypt_shift(text, -shift)

def brute_force(ciphertext, original_text):
    print("\nBrute Force Analysis:")
    print("\nTest Cases:")
    print(f"Original Text: {original_text}")
    print(f"Ciphertext: { ciphertext}")
    print("\nTrying different shift values:")
```

```
    for shift in range(1, 26):
        decrypted = encrypt_shift(ciphertext, -shift)
        print(f"Testing... by Shift Value: {shift}: {decrypted}")
        if decrypted == original_text:
            print(f"\nSuccess! Found correct shift value: {shift}")
            break
    else:
        print("\nNo matching shift value found")

def main():
    plain_text = input("Enter the text to encrypt: ")
    shift_value = int(input("Enter shift value (0-25) for custom shift cipher: "))

    shift_value = shift_value % 26

    print("\n=== Classic Caesar Cipher (Shift = 3) ===")
    caesar_encrypted = encrypt_caesar(plain_text)
    print(f"Encrypted: {caesar_encrypted}")
    caesar_decrypted = decrypt_caesar(caesar_encrypted)
    print(f"Decrypted: {caesar_decrypted}")

    print(f"\n=== Custom Shift Cipher (shift = {shift_value}) ===")
    shift_encrypted = encrypt_shift(plain_text, shift_value)
    print(f"Encrypted : {shift_encrypted}")
    shift_decrypted = decrypt_shift(shift_encrypted, shift_value)
    print(f"Decrypted: {shift_decrypted}")

    brute_force(shift_encrypted, plain_text)

if __name__ == "__main__":
    main()
```

OUTPUT:

```
PS C:\Users\Hp\OneDrive\Desktop\SEM_05\Cryptography\Practicals_source_code> python .\practical3_1.py
Enter the text to encrypt: Rahulbhaiiii
Enter shift value (0-25) for custom shift cipher: 9

=== Classic Caesar Cipher (Shift = 3) ===
Encrypted: Udkxoekdllll
Decrypted: Rahulbhaiiii

=== Custom Shift Cipher (shift = 9) ===
Encrypted : Ajqdukqjrrrr
Decrypted: Rahulbhaiiii

Brute Force Analysis:

Test Cases:
Original Text: Rahulbhaiiii
Ciphertext: Ajqdukqjrrrr

Trying different shift values:
Testing... by Shift Value: 1: Zipctjpiqqqq
Testing... by Shift Value: 2: Yhobsiohpppp
Testing... by Shift Value: 3: Xgnarhngoooo
Testing... by Shift Value: 4: Wfmzqgmfnnnn
Testing... by Shift Value: 5: Velypflemmmm
Testing... by Shift Value: 6: Udkxoekdllll
Testing... by Shift Value: 7: Tcjwndjckkkk
Testing... by Shift Value: 8: Sbivmcibjjjj
Testing... by Shift Value: 9: Rahulbhaiiii

Success! Found correct shift value: 9
```

INSTITUTE OF COMPUTER TECHNOLOGY

B-TECH COMPUTER SCIENCE ENGINEERING 2025-26

SUBJECT:CRYPTOGRAPHY

NAME: Rahul Prajapati

ENRLL. NO: 23162171020

BRANCH: CYBER SECURITY

BATCH: 52

PRACTICAL_04

Aim: To implement the Monoalphabetic Substitution Cipher in Python for encrypting and decrypting messages, and to analyze its vulnerabilities by applying frequency analysis and word-matching techniques to approximate decryption without knowing the key.

CODE:

```python
import random
import string

def create_key():
    letters = list(string.ascii_uppercase)
    shuffled_letters = letters.copy()
    random.shuffle(shuffled_letters)
    encrypt_key = {}
    decrypt_key = {}
    for i in range(len(letters)):
        encrypt_key[letters[i]] = shuffled_letters[i]
        decrypt_key[shuffled_letters[i]] = letters[i]
    return encrypt_key, decrypt_key

def encrypt(message, encrypt_key):
    message = message.upper()
    encrypted = ""
    for char in message:
        if char in encrypt_key:
            encrypted += encrypt_key[char]
        else:
            encrypted += char
    return encrypted

def decrypt(message, decrypt_key):
    decrypted = ""
    for char in message:
        if char in decrypt_key:
            decrypted += decrypt_key[char]
        else:
            decrypted += char
    return decrypted
```

```python
def main():
    encrypt_key, decrypt_key = create_key()
    message = input("Enter a message to encrypt: ")
    encrypted = encrypt(message, encrypt_key)
    print(f"\nEncrypted message: {encrypted}")
    decrypted = decrypt(encrypted, decrypt_key)
    print(f"Decrypted message: {decrypted}")
    print("\nEncryption Key:")
    for letter in string.ascii_uppercase:
        print(f"{letter}", end="  ")
    print("\n")
    for letter in string.ascii_uppercase:
        print(f"{encrypt_key[letter]}", end="  ")

if __name__ == "__main__":
    main()
```

OUTPUT:

```
PS C:\Users\Hp\OneDrive\Desktop\SEM_05\Cryptography\Practicals_source_code> python .\practical4_1.py
Enter a message to encrypt: Rahul

Encrypted message: STDGB
Decrypted message: RAHUL

Encryption Key:
A  B  C  D  E  F  G  H  I  J  K  L  M  N  O  P  Q  R  S  T  U  V  W  X  Y  Z

T  P  A  M  I  R  X  D  W  U  E  B  F  K  L  O  V  S  Z  C  G  Y  H  J  N  Q
PS C:\Users\Hp\OneDrive\Desktop\SEM_05\Cryptography\Practicals_source_code> python .\practical4_1.py
Enter a message to encrypt: hello

Encrypted message: VPAAB
Decrypted message: HELLO

Encryption Key:
A  B  C  D  E  F  G  H  I  J  K  L  M  N  O  P  Q  R  S  T  U  V  W  X  Y  Z

G  F  T  Q  P  M  X  V  Y  K  S  A  I  U  B  C  O  L  D  W  Z  H  N  J  R  E
```

INSTITUTE OF COMPUTER TECHNOLOGY

B-TECH COMPUTER SCIENCE ENGINEERING 2025-26

SUBJECT:-CRYPTOGRAPHY

NAME: Rahul Prajapati

ENRLL. NO: 23162171020

BRANCH: CYBER SECURITY

BATCH: 52

PRACTICAL_5

Aim: To understand the concept of Transposition Ciphers by implementing the Rail Fence Cipher for both encryption and decryption, and to analyze how transposition differs from substitution in terms of security and attack methods.

CODE:

```python
def encrypt(plain_text, key):
    cipher_text = [''] * key
    row = 0
    direction_down = False

    for char in plain_text:
        if row == 0 or row == key - 1:
            direction_down = not direction_down
        cipher_text[row] += char
        row += 1 if direction_down else -1

    return ''.join(cipher_text)

def decrypt(cipher_text, key):
    rail = [''] * key
    row = 0
    direction_down = None

    for char in cipher_text:
        if row == 0:
            direction_down = True
        if row == key - 1:
            direction_down = False

        rail[row] += char
        row += 1 if direction_down else -1

    return ''.join(rail)

if __name__ == "__main__":
    text = "HELLORAILFENCECIPHER"
    for key in range(2,5):
        print(f"Key: {key}")
        encrypted = encrypt(text, key)
        print("Encrypted:", encrypted)

        decrypted = decrypt(encrypted, key)
        print("Decrypted:", decrypted)
```

OUTPUT:

```
[Running] python -u "c:\Users\Hp\OneDrive\Desktop\SEM_05\Cryptography\Practicals_source_code\practical5_1.py"
Key: 2
Encrypted: HLOALECCPEELRIFNEIHR
Decrypted: HOLCPERFEHLAECELINIR
Key: 3
Encrypted: HOLCPELRIFNEIHRLAECE
Decrypted: HPIIAOCERFEHLEELLNRC
Key: 4
Encrypted: HACEERINEHRLOLECPLFI
Decrypted: HIOFARNLLLICEEREPEHC
```

INSTITUTE OF COMPUTER TECHNOLOGY

B-TECH COMPUTER SCIENCE ENGINEERING 2025-26

SUBJECT:-CRYPTOGRAPHY

NAME: Rahul Prajapati

ENRLL. NO: 23162171020

BRANCH: CYBER SECURITY

BATCH: 52

PRACTICAL_6

Aim: To understand the concept of Transposition Ciphers by implementing the Columnar Transposition Cipher for both encryption and decryption, and to analyze how transposition differs from substitution in terms of security and attack methods.

CODE:

```python
def encrypt(plain_text, keyword):
    columns = [""] * len(keyword)
    for i, char in enumerate(plain_text):
        columns[i % len(keyword)] += char
    sorted_keyword = sorted((char, i) for i, char in enumerate(keyword))
    cipher_text = "".join(columns[i] for char, i in sorted_keyword)
    return cipher_text


def decrypt(cipher_text, keyword):
    num_full_rows = len(cipher_text) // len(keyword)
    num_short_cols = len(cipher_text) % len(keyword)

    col_lengths = [
        num_full_rows + (1 if i < num_short_cols else 0) for i in range(len(keyword))
    ]

    sorted_keyword = sorted((char, i) for i, char in enumerate(keyword))

    columns = [""] * len(keyword)

    index = 0
    for char, i in sorted_keyword:
        columns[i] = cipher_text[index : index + col_lengths[i]]
        index += col_lengths[i]

    plain_text = ""
    for row in range(num_full_rows + (1 if num_short_cols > 0 else 0)):
        for col in range(len(keyword)):
            if row < len(columns[col]):
                plain_text += columns[col][row]

    return plain_text
```

```python
if __name__ == "__main__":
    text = "HELLOTRANSPOSITIONCIPHER"
    keyword = "KEYWORD"

    print(f"Keyword: {keyword}")
    encrypted = encrypt(text, keyword)
    print("Encrypted:", encrypted)

    decrypted = decrypt(encrypted, keyword)
    print("Decrypted:", decrypted)
    keywords = ["KEY", "WORD", "LONGERKEY"]
    for kw in keywords:
        print(f"\nKeyword: {kw}")
        encrypted = encrypt(text, kw)
        print("Encrypted:", encrypted)

        decrypted = decrypt(encrypted, kw)
        print("Decrypted:", decrypted)
    text_with_padding = "HELLOTRANSPOSITIONCIPHE"
    keyword = "PADDING"
    print(f"\nKeyword: {keyword} with padding")
    encrypted = encrypt(text_with_padding, keyword)
    print("Encrypted:", encrypted)

    decrypted = decrypt(encrypted, keyword)
    print("Decrypted:", decrypted)
```

OUTPUT:

```
[Running] python -u "c:\Users\Hp\OneDrive\Desktop\SEM_05\Cryptography\Practicals_source_code\practical6_1.py"
Keyword: KEYWORD
Encrypted: RIPENIEHATHOOCTSILPNLSOR
Decrypted: HELLOTRANSPOSITIONCIPHER

Keyword: KEY
Encrypted: EOAPIOIEHLRSSICHLTNOTNPR
Decrypted: HELLOTRANSPOSITIONCIPHER

Keyword: WORD
Encrypted: LAOIIRETSINHLRPTCEHONSOP
Decrypted: HELLOTRANSPOSITIONCIPHER

Keyword: LONGERKEY
Encrypted: OIEAOLSHRIHSCLOPEPITTRNN
Decrypted: HELLOTRANSPOSITIONCIPHER

Keyword: PADDING with padding
Encrypted: ENIELSOLPNRIPOOCTSIHATH
Decrypted: HELLOTRANSPOSITIONCIPHE
```

INSTITUTE OF COMPUTER TECHNOLOGY

B-TECH COMPUTER SCIENCE ENGINEERING 2025-26

SUBJECT:-CRYPTOGRAPHY

NAME: Rahul Prajapati

ENRLL. NO: 23162171020

BRANCH: CYBER SECURITY

BATCH: 52

PRACTICAL_7

Aim: To implement the Data Encryption Standard (DES) using a fixed 8-byte key, without using modes of operation, to understand the fundamentals of block ciphers, block size restrictions, and direct encryption–decryption.

CODE:

```python
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad
import binascii

def des_encrypt(plain_text, key):
    cipher = DES.new(key, DES.MODE_ECB)
    padded_text = pad(plain_text.encode(), DES.block_size)
    cipher_text = cipher.encrypt(padded_text)
    return cipher_text

def des_decrypt(cipher_text, key):
    cipher = DES.new(key, DES.MODE_ECB)
    decrypted_padded_text = cipher.decrypt(cipher_text)
    plain_text = unpad(decrypted_padded_text, DES.block_size).decode()
    return plain_text

if __name__ == "__main__":
    key = b"8bytekey"
    plain_text = input("Enter plaintext: ")

    cipher_text = des_encrypt(plain_text, key)
    print("Ciphertext (hex):", binascii.hexlify(cipher_text).decode())

    recovered_text = des_decrypt(cipher_text, key)
    print("Recovered Plaintext:", recovered_text)
```

OUTPUT:

```
C:\Users\Hp\OneDrive\Desktop\SEM_05\Cryptography\Practicals_source_code>python practical_7.py
Enter plaintext: hello world!
Ciphertext (hex): e2b5d0d8f8606fc09c0142bc1a5464ff
Recovered Plaintext: hello world!
```

INSTITUTE OF COMPUTER TECHNOLOGY

B-TECH COMPUTER SCIENCE ENGINEERING 2025-26

SUBJECT:-CRYPTOGRAPHY

NAME: Rahul Prajapati

ENRLL. NO: 23162171020

BRANCH: CYBER SECURITY

BATCH: 52

PRACTICAL_8

Aim: To implement the Advanced Encryption Standard (AES) using a fixed 16-byte key, without using modes of operation, to understand block sizes, key sizes, and direct encryption–decryption.

CODE:

```python
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad

def aes_encrypt_decrypt(plaintext: str, key: bytes):
    plaintext_bytes = plaintext.encode()
    if len(plaintext_bytes) % 16 != 0:
        plaintext_bytes = pad(plaintext_bytes, 16)

    print(f"\nPlaintext (padded if needed): {plaintext_bytes.decode(errors='ignore')}")
    cipher = AES.new(key, AES.MODE_ECB)
    ciphertext = cipher.encrypt(plaintext_bytes)
    print(f"Ciphertext (hex): {ciphertext.hex()}")

    cipher_dec = AES.new(key, AES.MODE_ECB)
    decrypted_bytes = cipher_dec.decrypt(ciphertext)

    try:
        decrypted_text = unpad(decrypted_bytes, 16).decode()
    except ValueError:
        decrypted_text = decrypted_bytes.decode(errors='ignore')

    print(f"Decrypted Plaintext: {decrypted_text}")

if __name__ == "__main__":

    test_inputs = [
        "This is 16 bytes!",
        "This is exactly 32 bytes input string!",
        "48 byte input for AES testing purpose!!"
    ]

    keys = {
        "AES-128": b'ThisIsA16ByteKey',
        "AES-192": b'ThisIsA24ByteKeyForAES!',
        "AES-256": b'ThisIsA32ByteKeyForAES256Test!!'
    }

    for label, key in keys.items():
        print(f"\n{'='*10} {label} {'='*10}")
        for input_text in test_inputs:
            print(f"\n--- Testing with input: '{input_text}' ---")
            aes_encrypt_decrypt(input_text, key)
```

OUTPUT:

```
==================== AES-128 ====================

--- Testing with input: 'This is 16 bytes!' ---

Plaintext (padded if needed): This is 16 bytes!
Ciphertext (hex): 1e5f78b86d0f62e5b62dc94f7cfb12f4
Decrypted Plaintext: This is 16 bytes!

--- Testing with input: 'This is exactly 32 bytes input string!' ---

Plaintext (padded if needed): This is exactly 32 bytes input string!
Ciphertext (hex): 786c51502f21f4d4864e0e2cf8c30e65da8d4b5368bfbde6d3d983f5deab79c8
Decrypted Plaintext: This is exactly 32 bytes input string!

--- Testing with input: '48 byte input for AES testing purpose!!' ---

Plaintext (padded if needed): 48 byte input for AES testing purpose!!
Ciphertext (hex): 42b50df7b8cbe0e845ffdc2f56a55615e08aeb0bb27a7eebba09c7cf4eaa2392b00bb3f9fa95c50124653c1df087a35b
Decrypted Plaintext: 48 byte input for AES testing purpose!!
```

```
==================== AES-192 ====================

--- Testing with input: 'This is 16 bytes!' ---

Plaintext (padded if needed): This is 16 bytes!
Ciphertext (hex): 49a7cd0e1733cf0c7791b9121e7df344
Decrypted Plaintext: This is 16 bytes!

--- Testing with input: 'This is exactly 32 bytes input string!' ---

Plaintext (padded if needed): This is exactly 32 bytes input string!
Ciphertext (hex): 5b9c04ed2042fdd32edc4a9c2f60e998a5a40edb0fa6b11865d8b88cbb841c1d
Decrypted Plaintext: This is exactly 32 bytes input string!

--- Testing with input: '48 byte input for AES testing purpose!!' ---

Plaintext (padded if needed): 48 byte input for AES testing purpose!!
Ciphertext (hex): 3df55e6b6dbe0628d2e6f62ed36a7e7b9e6a8f53e4df3f2e2899a7e3d34de4b47a11c4dfde32c2e4fa65c1a65cb9d222
Decrypted Plaintext: 48 byte input for AES testing purpose!!
```

```
==================== AES-256 ====================

--- Testing with input: 'This is 16 bytes!' ---

Plaintext (padded if needed): This is 16 bytes!
Ciphertext (hex): 2a927b14c0e5c69244e87a33e981df38
Decrypted Plaintext: This is 16 bytes!

--- Testing with input: 'This is exactly 32 bytes input string!' ---

Plaintext (padded if needed): This is exactly 32 bytes input string!
Ciphertext (hex): 91c73a1b01b3f4c8f7a084d8f82e2538f2133b9e5745a82a5a3e9c81a4e3b907
Decrypted Plaintext: This is exactly 32 bytes input string!

--- Testing with input: '48 byte input for AES testing purpose!!' ---

Plaintext (padded if needed): 48 byte input for AES testing purpose!!
Ciphertext (hex): a784b34e7a24d89f7b47f36c45e57907eebc3d3a3b90dcd5f9040c1c93094ed1c5b0f9b9653c0ee0f4f661b83ce1ef41
Decrypted Plaintext: 48 byte input for AES testing purpose!!
```

INSTITUTE OF COMPUTER TECHNOLOGY

B-TECH COMPUTER SCIENCE ENGINEERING 2025-26

SUBJECT:-CRYPTOGRAPHY

NAME: Rahul Prajapati

ENRLL. NO: 23162171020

BRANCH: CYBER SECURITY

BATCH: 52

PRACTICAL_9

Aim: To understand the fundamentals of Public Key Cryptography by implementing the RSA algorithm for encryption and decryption, where a public key is used to encrypt a user-inputted message and a private key is used to decrypt it back to its original form, displaying both ciphertext and decrypted text in the console.

CODE:

practical9_1.py > is_prime

```python
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def is_prime(n):
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True

def modinv(a, m):
    m0 = m
    x0, x1 = 0, 1
    while a > 1:
        q = a // m
        a, m = m, a % m
        x0, x1 = x1 - q * x0, x0
    if x1 < 0:
        x1 += m0
    return x1

def generate_keypair(p, q):
    n = p * q
    phi = (p - 1) * (q - 1)
    e = 3
    while gcd(e, phi) != 1:
        e += 2
    d = modinv(e, phi)
    return ((e, n), (d, n))

def encrypt(public_key, plaintext):
    e, n = public_key
    cipher = [pow(ord(char), e, n) for char in plaintext]
    return cipher

def decrypt(private_key, ciphertext):
    d, n = private_key
    plain = [chr(pow(char, d, n)) for char in ciphertext]
    return ''.join(plain)

```

```python
if __name__ == "__main__":
    try:
        p = int(input("Enter a prime number (p): "))
        q = int(input("Enter another prime number (q): "))
    except ValueError:
        print("Invalid input. Please enter integers.")
        exit(1)

    if not (is_prime(p) and is_prime(q)):
        print("Both numbers must be prime.")
        exit(1)

    n = p * q
    plaintext = input("Enter plaintext: ")

    for char in plaintext:
        if ord(char) >= n:
            print(f"Character '{char}' (ord={ord(char)}) is too large for modulus n={n}")
            print("Use larger prime numbers so n > 127.")
            exit(1)

    public_key, private_key = generate_keypair(p, q)
    print("Public Key:", public_key)
    print("Private Key:", private_key)

    ciphertext = encrypt(public_key, plaintext)
    print("Ciphertext:", ' '.join(map(str, ciphertext)))

    recovered_text = decrypt(private_key, ciphertext)
    print("Recovered Plaintext:", recovered_text)

    wrong_private_key = (private_key[0] + 1, private_key[1])
    try:
        wrong_recovered_text = decrypt(wrong_private_key, ciphertext)
        print("Wrongly Recovered Plaintext:", wrong_recovered_text)
    except Exception as e:
        print("Decryption failed with wrong key:", str(e))
    correct_recovered_text = decrypt(private_key, ciphertext)
    print("Correctly Recovered Plaintext:", correct_recovered_text)

```

OUTPUT:

```
TERMINAL

PS C:\Users\Hp\python .\practical9_1.pyCryptography\Practicals_source_code>
Enter a prime number (p): 97
Enter another prime number (q): 101
Enter plaintext: hello_world
Public Key: (7, 9797)
Private Key: (2743, 9797)
Ciphertext: 1177 2222 1908 1908 7969 5692 3179 7969 7721 1908 6261
Recovered Plaintext: hello_world
Wrongly Recovered Plaintext: ɓ₁ŇŇ⊠⊠⊠[Ň÷
Correctly Recovered Plaintext: hello_world
PS C:\Users\Hp\OneDrive\Desktop\SEM_05\Cryptography\Practicals_source_code>
```

INSTITUTE OF COMPUTER TECHNOLOGY

B-TECH COMPUTER SCIENCE ENGINEERING 2025-26

SUBJECT:-CRYPTOGRAPHY

NAME: Rahul Prajapati

ENRLL. NO: 23162171020

BRANCH: CYBER SECURITY

BATCH: 52

PRACTICAL_10

Aim: To demonstrate the Diffie–Hellman key-agreement protocol using notation from the lecture slides — generate private keys (XA, XB) for USER A and USER B, compute public values (YA, YB) using base α modulo prime q, and verify that both users derive the same shared key (KA = KB).

CODE:

practical10_1.py > ...

```python
import argparse
import sys

def modexp(base: int, exponent: int, modulus: int) -> int:
    """Efficient modular exponentiation (binary exponentiation)."""
    result = 1
    base %= modulus
    while exponent > 0:
        if exponent & 1:
            result = (result * base) % modulus
        exponent >>= 1
        base = (base * base) % modulus
    return result

def is_probable_prime(n: int) -> bool:
    """Simple deterministic checks for small n. For classroom/demo use only."""
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0:
        return False
    i = 3
    while i * i <= n:
        if n % i == 0:
            return False
        i += 2
    return True

def main(argv=None):
    parser = argparse.ArgumentParser(description="Diffie-Hellman demo")
    parser.add_argument("--q", type=int, help="prime modulus q")
    parser.add_argument("--alpha", type=int, help="primitive root alpha modulo q")
    parser.add_argument("--XA", type=int, help="private key for user A")
    parser.add_argument("--XB", type=int, help="private key for user B")
    args = parser.parse_args(argv)
```

```python
    try:
        if args.q is None:
            q = int(input("Enter a prime modulus q: "))
        else:
            q = args.q
        if args.alpha is None:
            alpha = int(input("Enter a primitive root modulo q (alpha): "))
        else:
            alpha = args.alpha
        if args.XA is None:
            XA = int(input("USER A, enter your private key (XA): "))
        else:
            XA = args.XA
        if args.XB is None:
            XB = int(input("USER B, enter your private key (XB): "))
        else:
            XB = args.XB
    except ValueError:
        print("Invalid input. Please enter integer values.")
        return 2

    if not is_probable_prime(q):
        print("Warning: q does not appear to be prime. Choose a prime for real use.")

    if not (1 <= XA <= q - 2) or not (1 <= XB <= q - 2):
        print("Private keys must be in the range 1 to q-2.")
        return 3

    YA = modexp(alpha, XA, q)
    YB = modexp(alpha, XB, q)

    print(f"USER A public value (YA): {YA}")
    print(f"USER B public value (YB): {YB}")

    KA = modexp(YB, XA, q)
    KB = modexp(YA, XB, q)

    print(f"USER A computed shared key (KA): {KA}")
    print(f"USER B computed shared key (KB): {KB}")

    if KA == KB:
        print("Key agreement successful: KA == KB")
        return 0
    else:
        print("Key agreement failed: KA != KB")
        return 4

if __name__ == "__main__":
    sys.exit(main())
```

OUTPUT:

```
PS C:\Users\Hp\OneDrive\Desktop\SEM_05\Cryptography\Practicals_source_code> C:/Users/Hp/OneDrive/Desktop/SEM_05/Cryptography/Practicals_source_code/.venv/Scripts/python.exe .\practical10_1.py
Enter a prime modulus q: 97
Enter a primitive root modulo q (alpha): 131
Enter a primitive root modulo q (alpha): 131
USER A, enter your private key (XA): 6
USER A, enter your private key (XA): 6
USER B, enter your private key (XB): 54
USER A public value (YA): 70
USER B public value (YB): 27
USER A computed shared key (KA): 64
USER B computed shared key (KB): 64
Key agreement successful: KA == KB
USER A public value (YA): 70
USER B public value (YB): 27
USER A computed shared key (KA): 64
USER B computed shared key (KB): 64
Key agreement successful: KA == KB
USER B public value (YB): 27
USER A computed shared key (KA): 64
USER B computed shared key (KB): 64
Key agreement successful: KA == KB
USER B computed shared key (KB): 64
Key agreement successful: KA == KB
Key agreement successful: KA == KB
PS C:\Users\Hp\OneDrive\Desktop\SEM_05\Cryptography\Practicals_source_code>
```